NeuroMIMO

RESEARCH PACK

Problem Statement • Literature Survey • Reference Papers • Sample Paper • Build & Feature Prompts

Companion to the NeuroMIMO project documentation: deep learning-assisted channel estimation for 2×2 MIMO-OFDM systems.

October 25, 2026

Contents

1. Problem Statement 2
2. Literature Survey 4
3. Reference Research Papers 6
4. Sample Research Paper 9
5. Prompts to Build and Extend the Project 12

1. Problem Statement

Background

Modern wireless standards such as 5G NR and Wi-Fi 6/7 combine Multiple-Input Multiple-Output (MIMO) antennas with Orthogonal Frequency-Division Multiplexing (OFDM) [1, 5, 34]. MIMO sends several data streams at once over the same frequency band, and OFDM splits a wideband channel into many narrow subcarriers that each see a simple, flat channel. To separate the streams and undo the distortion on each subcarrier, the receiver must know the channel frequency response **H**, so channel estimation sits at the heart of every MIMO-OFDM receiver.

Receivers estimate **H** from known *pilot* symbols inserted between the data. The two classical estimators are Least Squares (LS) and Linear Minimum Mean Square Error (LMMSE) [8, 11]. LS is simple and fast but amplifies noise at low signal-to-noise ratio (SNR). LMMSE suppresses noise well but needs the channel correlation matrix and noise variance, which are hard to track in practice, and it requires a costly matrix inversion.

Problem

How accurately, how reliably, and at what computational cost can a lightweight deep neural network estimate the channel of a 2×2 MIMO-OFDM link from pilot observations, compared with LS and LMMSE, across low to high SNR, and how well does it generalise to channels it was not trained on?

Published deep learning estimators [19, 20] often report gains in narrow settings and seldom report statistical confidence, latency, or behaviour on unseen channels. Students and researchers also lack an open, reproducible tool that runs the classical and learned estimators side by side on *identical* channel realisations and shows the results interactively.

Objectives

- Build a vectorised 2×2 MIMO-OFDM baseband simulator: QPSK/16-QAM mapping, 64-point FFT, 16-sample cyclic prefix, orthogonal comb pilots, AWGN and 4-tap Rayleigh multipath channels.
- Implement LS (with DFT interpolation) and LMMSE estimators as classical baselines, plus a perfect-CSI genie bound.
- Design and train a residual multilayer perceptron (MLP) that refines the LS estimate, with a leakage-free train/validation/test split by channel realisation.
- Compare all estimators by NMSE, BER and SER over 0–30 dB SNR with 95% confidence intervals, and measure inference latency.
- Test generalisation to mismatched power-delay profiles, longer channels and different pilot densities.
- Deliver the work as a FastAPI back end and a React dashboard so results are reproducible and can be explored interactively.

Scope and Assumptions

- Software simulation only; no radio hardware. Synthetic channels with an exponential power-delay profile and block fading (the channel is constant within one OFDM frame).
- 2 transmit and 2 receive antennas, 64 subcarriers, perfect time and frequency synchronisation.

- The DNN runs on CPU; results must be reproducible from fixed random seeds.

Expected Outcomes and Success Criteria

- A working simulator whose LS and LMMSE results follow theory (for example, NMSE improving by about 10 dB per 10 dB of SNR for LS).
- A trained DNN with measured NMSE/BER versus SNR, honestly reported where it wins and where it loses.
- Latency below 1 ms per frame on a CPU for every estimator.
- An automated test suite and a dashboard that reproduce every figure in the report.

2. Literature Survey

This survey moves from the foundations of MIMO and OFDM, through classical pilot-aided channel estimation, to deep learning for the physical layer and learned channel estimators. It ends with the research gap that NeuroMIMO addresses. Numbers in square brackets refer to Section 3.

2.1 MIMO and OFDM Foundations

Telatar [1] and Foschini and Gans [2] showed that the capacity of a rich-scattering channel grows roughly linearly with the smaller of the number of transmit and receive antennas, which is the basis of spatial multiplexing. Alamouti [3] gave a simple two-antenna transmit diversity scheme. OFDM was made practical by Weinstein and Ebert [4], who implemented multicarrier modulation with the DFT; the cyclic prefix turns the linear channel convolution into a circular one, so each subcarrier sees a single complex gain. Hwang *et al.* [5] survey OFDM in modern standards, and the textbooks [6, 7] cover both topics in depth.

2.2 Classical Pilot-Aided Channel Estimation

van de Beek *et al.* [8] compared LS and LMMSE estimators for OFDM and showed that LMMSE greatly reduces error at low SNR at a large computational cost. Edfors *et al.* [9] reduced that cost with a low-rank approximation based on the singular value decomposition. Li, Cimini and Sollenberger [10] exploited the time-frequency correlation of the channel for robust estimation in fast fading. Morelli and Mengali [11] compared pilot-aided methods systematically, and Coleri *et al.* [12] studied block-type versus comb-type pilot arrangements and interpolation methods. Ozdemir and Arslan [13] give a broad survey. For MIMO, Biguesh and Gershman [14] analysed LS and LMMSE trade-offs and optimal training signals. The estimation theory behind these methods is covered by Kay [15].

Takeaway: LS needs no prior knowledge but suffers from noise enhancement; LMMSE is optimal among linear estimators but depends on accurate channel statistics and a matrix inverse. This trade-off motivates data-driven estimators that learn the prior from examples.

2.3 Deep Learning for the Physical Layer

O'Shea and Hoydis [16] framed the transmitter, channel and receiver as an autoencoder that can be trained end to end. Wang *et al.* [17] and Qin *et al.* [18] reviewed opportunities and open challenges, noting that learned blocks are most useful where models are inaccurate or optimal algorithms are too complex, and warning about generalisation and the cost of training data.

2.4 Deep Learning Channel Estimation

Ye, Li and Juang [19] trained a fully connected network to recover transmitted symbols directly from received OFDM signals, outperforming LS and matching LMMSE with fewer pilots and without a cyclic prefix. Soltani *et al.* [20] treated the pilot grid as a low-resolution image and used super-resolution and denoising CNNs (ChannelNet) to reconstruct the full channel. Neumann, Wiese and Utschick [21] derived a neural network structure from the MMSE estimator itself, giving a learned estimator with lower complexity. For millimetre-wave massive MIMO, He *et al.* [22] used a learned denoising network and Dong *et al.* [23] used deep CNNs that exploit correlation across frequency and time. Wen, Shih and Jin [24] compressed channel state information for feedback (CsiNet), and Mashhadi and Gündüz [25] jointly learned pilot placement and channel estimation for MIMO-OFDM.

2.5 Neural Receivers

More recent work replaces several receiver blocks at once. DeepRx [26] is a fully convolutional receiver that maps received resource grids to bit likelihoods. Ait Aoudia and Hoydis [27] learned OFDM receivers end to end and showed that pilots can be reduced or removed. The open-source Sionna library [28] provides differentiable link-level simulation for such research.

2.6 Training and Evaluation Methods

Residual connections [29] let a network learn a correction to its input, which suits refining an existing LS estimate. Adam [30] is the standard optimiser. For Monte Carlo BER results, the Wilson score interval [31] gives reliable confidence intervals for binomial proportions, and the "rule of three" [32] bounds the true error rate when no errors are observed. Time-varying channels are commonly modelled with the Jakes Doppler spectrum [33].

2.7 Summary of Key Works

Ref.	Approach	Setting	Key finding / relevance
[8]	LS vs LMMSE	SISO-OFDM	LMMSE wins at low SNR at high cost; NeuroMIMO's baselines
[12]	Pilot arrangements	SISO-OFDM	Comb pilots + interpolation; basis for pilot spacing study
[14]	LS/LMMSE for MIMO	MIMO training	Estimator trade-offs and optimal training
[19]	Fully connected DNN	SISO-OFDM	DNN matches LMMSE with fewer pilots
[20]	Image super-resolution CNN	OFDM grid	Treats pilots as an image; strong denoising
[21]	MMSE-structured network	Generic	Learned estimator close to MMSE, lower cost
[23]	Deep CNN	mmWave massive MIMO	Exploits frequency/time correlation
[25]	Joint pilot design + DNN	MIMO-OFDM	Fewer pilots with learned placement
[27]	End-to-end neural receiver	OFDM	Pilot reduction; full-receiver learning

Table 1: Representative works and their relevance to this project.

2.8 Research Gap and How NeuroMIMO Addresses It

- **Few honest high-SNR results.** Many papers emphasise low-SNR gains. NeuroMIMO reports the full 0–30 dB range and shows that its DNN falls clearly behind LS above about 15 dB (an error floor).
- **Little statistical rigour.** NeuroMIMO attaches Wilson intervals to every BER point and uses the rule of three for zero-error points.
- **Generalisation is rarely tested.** NeuroMIMO evaluates mismatched delay profiles, a longer 6-tap channel and different pilot spacings.
- **Hard to reproduce.** NeuroMIMO runs every estimator on identical channel realisations from fixed seeds, with an automated test suite and an interactive dashboard.

3. Reference Research Papers

These are real, published works, grouped by topic in the order they are cited. Use them as the reference list for your report and as a reading list.

> **Before you submit:** this list was compiled from memory, not from a live database search. Check each entry on IEEE Xplore or Google Scholar, confirm the volume, issue and page numbers, and add the DOI. Read at least the abstract of every paper you cite.

MIMO and OFDM foundations

[1] I. E. Telatar, “Capacity of multi-antenna Gaussian channels,” *European Trans. Telecommunications*, vol. 10, no. 6, pp. 585–595, 1999.

[2] G. J. Foschini and M. J. Gans, “On limits of wireless communications in a fading environment when using multiple antennas,” *Wireless Personal Communications*, vol. 6, no. 3, pp. 311–335, 1998.

[3] S. M. Alamouti, “A simple transmit diversity technique for wireless communications,” *IEEE J. Sel. Areas Commun.*, vol. 16, no. 8, pp. 1451–1458, Oct. 1998.

[4] S. B. Weinstein and P. M. Ebert, “Data transmission by frequency-division multiplexing using the discrete Fourier transform,” *IEEE Trans. Commun. Technol.*, vol. 19, no. 5, pp. 628–634, Oct. 1971.

[5] T. Hwang, C. Yang, G. Wu, S. Li, and G. Y. Li, “OFDM and its wireless applications: A survey,” *IEEE Trans. Veh. Technol.*, vol. 58, no. 4, pp. 1673–1694, May 2009.

[6] D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*. Cambridge, U.K.: Cambridge Univ. Press, 2005.

[7] A. Goldsmith, *Wireless Communications*. Cambridge, U.K.: Cambridge Univ. Press, 2005.

Classical pilot-aided channel estimation

[8] J.-J. van de Beek, O. Edfors, M. Sandell, S. K. Wilson, and P. O. Börjesson, “On channel estimation in OFDM systems,” in *Proc. IEEE 45th Veh. Technol. Conf.*, 1995, pp. 815–819.

[9] O. Edfors, M. Sandell, J.-J. van de Beek, S. K. Wilson, and P. O. Börjesson, “OFDM channel estimation by singular value decomposition,” *IEEE Trans. Commun.*, vol. 46, no. 7, pp. 931–939, Jul. 1998.

[10] Y. Li, L. J. Cimini, and N. R. Sollenberger, “Robust channel estimation for OFDM systems with rapid dispersive fading channels,” *IEEE Trans. Commun.*, vol. 46, no. 7, pp. 902–915, Jul. 1998.

[11] M. Morelli and U. Mengali, “A comparison of pilot-aided channel estimation methods for OFDM systems,” *IEEE Trans. Signal Process.*, vol. 49, no. 12, pp. 3065–3073, Dec. 2001.

[12] S. Coleri, M. Ergen, A. Puri, and A. Bahai, “Channel estimation techniques based on pilot arrangement in OFDM systems,” *IEEE Trans. Broadcast.*, vol. 48, no. 3, pp. 223–229, Sep. 2002.

[13] M. K. Ozdemir and H. Arslan, “Channel estimation for wireless OFDM systems,” *IEEE Commun. Surveys Tuts.*, vol. 9, no. 2, pp. 18–48, 2007.

[14] M. Biguesh and A. B. Gershman, “Training-based MIMO channel estimation: A study of estimator tradeoffs and optimal training signals,” *IEEE Trans. Signal Process.*, vol. 54, no. 3, pp. 884–893, Mar. 2006.

[15] S. M. Kay, *Fundamentals of Statistical Signal Processing, Volume I: Estimation Theory*. Upper Saddle River, NJ, USA: Prentice Hall, 1993.

Deep learning for the physical layer

[16] T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 563–575, Dec. 2017.

[17] T. Wang, C.-K. Wen, H. Wang, F. Gao, T. Jiang, and S. Jin, "Deep learning for wireless physical layer: Opportunities and challenges," *China Communications*, vol. 14, no. 11, pp. 92–111, Nov. 2017.

[18] Z. Qin, H. Ye, G. Y. Li, and B.-H. F. Juang, "Deep learning in physical layer communications," *IEEE Wireless Commun.*, vol. 26, no. 2, pp. 93–99, Apr. 2019.

Deep learning channel estimation

[19] H. Ye, G. Y. Li, and B.-H. F. Juang, "Power of deep learning for channel estimation and signal detection in OFDM systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 1, pp. 114–117, Feb. 2018.

[20] M. Soltani, V. Pourahmadi, A. Mirzaei, and H. Sheikhzadeh, "Deep learning-based channel estimation," *IEEE Commun. Lett.*, vol. 23, no. 4, pp. 652–655, Apr. 2019.

[21] D. Neumann, T. Wiese, and W. Utschick, "Learning the MMSE channel estimator," *IEEE Trans. Signal Process.*, vol. 66, no. 11, pp. 2905–2917, Jun. 2018.

[22] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Deep learning-based channel estimation for beamspace mmWave massive MIMO systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 5, pp. 852–855, Oct. 2018.

[23] P. Dong, H. Zhang, G. Y. Li, I. S. Gaspar, and N. NaderiAlizadeh, "Deep CNN-based channel estimation for mmWave massive MIMO systems," *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 5, pp. 989–1000, Sep. 2019.

[24] C.-K. Wen, W.-T. Shih, and S. Jin, "Deep learning for massive MIMO CSI feedback," *IEEE Wireless Commun. Lett.*, vol. 7, no. 5, pp. 748–751, Oct. 2018.

[25] M. B. Mashhadi and D. Gündüz, "Pruning the pilots: Deep learning-based pilot design and channel estimation for MIMO-OFDM systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6315–6328, Oct. 2021.

Neural receivers and simulation tools

[26] M. Honkala, D. Korpi, and J. M. J. Huttunen, "DeepRx: Fully convolutional deep learning receiver," *IEEE Trans. Wireless Commun.*, vol. 20, no. 6, pp. 3925–3940, Jun. 2021.

[27] F. Ait Aoudia and J. Hoydis, "End-to-end learning for OFDM: From neural receivers to pilotless communication," *IEEE Trans. Wireless Commun.*, vol. 21, no. 2, pp. 1049–1063, Feb. 2022.

[28] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An open-source library for next-generation physical layer research," arXiv:2203.11854, 2022.

Machine learning, statistics and standards

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[31] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statistical Science*, vol. 16, no. 2, pp. 101–133, 2001.

[32] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983.

[33] W. C. Jakes, Ed., *Microwave Mobile Communications*. New York, NY, USA: Wiley, 1974.

[34] 3GPP, "NR; Physical channels and modulation," Technical Specification TS 38.211.

Suggested Reading Order

- **Start here:** [13] for a survey of OFDM channel estimation and [16] for deep learning in the physical layer.
- **Core baselines:** [8, 12, 14], which match NeuroMIMO's LS, LMMSE and comb-pilot design.
- **Closest to this project:** [19, 20, 21].
- **State of the art and future work:** [25, 26, 27, 28].

4. Sample Research Paper

A short paper in IEEE conference style, written from this project's own code and results. Use it as a starting draft: expand each section, add your own figures from `backend/results/`, and put your name and institution in the author line.

Residual Deep Learning for Pilot-Aided Channel Estimation in 2×2 MIMO-OFDM: An Honest Benchmark Against LS and LMMSE

Author Name, Department, Institution, email

Abstract

Accurate channel estimation is essential for MIMO-OFDM receivers. This paper evaluates a lightweight residual multilayer perceptron that refines the least-squares (LS) channel estimate of a 2×2 MIMO-OFDM link with 64 subcarriers and comb-type pilots. The network has 330,112 parameters and is compared with LS and linear MMSE (LMMSE) on identical Rayleigh channel realisations over 0–30 dB SNR, with 47,040 bits per SNR point and 95% Wilson confidence intervals. At 0 dB the network reaches −8.36 dB NMSE, matching LS (−8.38 dB) and about 4.3 dB better than LMMSE (−4.01 dB). Above 15 dB it shows an error floor near −24.76 dB, while LS reaches −38.64 dB at 30 dB. Mean inference latency is 0.49 ms per frame on a CPU. We discuss the causes of the floor and directions to remove it.

Index Terms—channel estimation, deep learning, MIMO, OFDM, residual network, least squares, MMSE.

I. Introduction

MIMO-OFDM underpins 5G NR and Wi-Fi [34, 5]. Receivers estimate the channel from pilots, classically with LS or LMMSE [8, 11]. Deep learning estimators [19, 20, 21] promise LMMSE-like accuracy without explicit channel statistics, but results are often reported over limited SNR ranges without confidence intervals. This paper contributes: (i) a reproducible 2×2 MIMO-OFDM benchmark in which all estimators see identical channels; (ii) a residual MLP estimator trained with a leakage-free split; and (iii) a full-range evaluation of accuracy, reliability, latency and generalisation.

II. System Model

Each transmit antenna sends N = 64 subcarriers with a 16-sample cyclic prefix. After FFT, the signal on subcarrier k at receive antenna r is

$$Y_r[k] = \sum_t H_{r,t}[k]\, X_t[k] + W_r[k]$$

where $H_{r,t}[k]$ is the frequency response of a 4-tap Rayleigh channel with an exponential power-delay profile and W is complex Gaussian noise. Transmit antenna 0 uses pilots on subcarriers {0, 16, 32, 48} and antenna 1 on {8, 24, 40, 56}, so pilots never overlap.

III. Channel Estimators

LS divides the received pilots by the known pilot symbols, $\hat{H}_{LS}[p] = Y[p] / X[p]$, and interpolates to all subcarriers with a DFT method. **LMMSE** applies $\hat{H} = \mathbf{R}_{HP}(\mathbf{R}_{PP} + \sigma_n^2\mathbf{I})^{-1}\hat{H}_{LS}$ using the exact channel correlation. The **residual DNN** computes

$$\hat{H}_{DNN} = \hat{H}_{LS} + f_\theta(\hat{H}_{LS})$$

where f_θ is an MLP ($512 \to 256 \to 128 \to 256 \to 512$) with LayerNorm, GELU and dropout [29], trained with Adam [30] on an MSE loss over 0–30 dB. Data are split by channel-realisation seed before feature extraction, so test channels are never seen during training.

IV. Simulation Setup

Parameter	Value
Antennas / subcarriers / CP	2×2 / 64 / 16
Modulation	QPSK
Channel	Rayleigh, 4 taps, exponential PDP (decay 1.5)
Pilot spacing	8 (4 pilots per transmit antenna)
Monte Carlo trials	15 per SNR point (47,040 bits)
SNR points	0, 5, 10, 15, 20, 25, 30 dB
Equalizer	MIMO MMSE

Table 2: Simulation parameters.

V. Results

SNR	LS NMSE	LMMSE NMSE	DNN NMSE	LS BER	DNN BER
0 dB	−8.38 dB	−4.01 dB	−8.36 dB	0.2310	0.2318
10 dB	−18.24 dB	−14.71 dB	−17.15 dB	0.0642	0.0659
20 dB	−28.95 dB	−27.30 dB	−23.06 dB	0.0092	0.0091
30 dB	−38.64 dB	−38.41 dB	−24.76 dB	0.0018	0.0066

Table 3: Estimation accuracy and bit error rate (Rayleigh, QPSK).

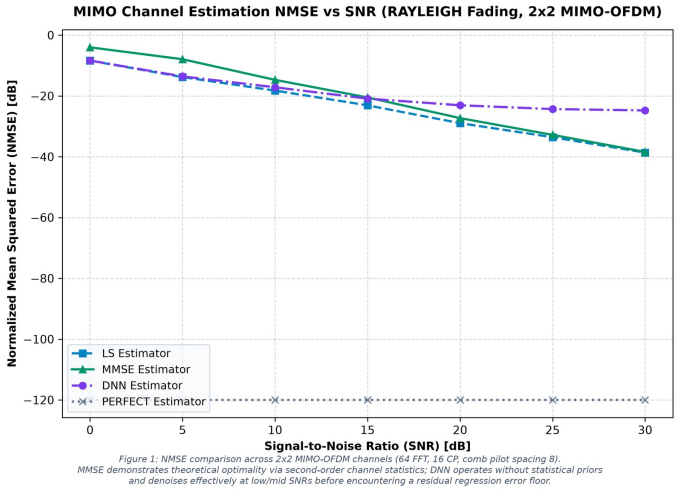


Fig. 1: NMSE versus SNR for all estimators.

At low SNR the DNN denoises as well as LS and better than LMMSE, without channel statistics. Above 15 dB its NMSE saturates near −24.76 dB, so at 30 dB it makes 311 bit errors against 85 for LS. Mean latency is 0.12 ms (LS), 0.35 ms (LMMSE) and 0.49 ms (DNN). With a 6-tap channel all estimators degrade to about -9.4 dB NMSE at 15 dB, because 4 pilots per antenna cannot resolve a 6-tap impulse response.

VI. Conclusion

A small residual MLP matches LS and beats LMMSE at low SNR but has an error floor at high SNR when one model is trained across the whole SNR range. Future work: feed SNR as an input or train per-SNR models, use 2D convolutional networks across time and frequency [20, 23], model Doppler with the Jakes spectrum [33], and learn pilot placement [25].

References: use the numbered list in Section 3.

5. Prompts to Build and Extend the Project

These prompts are written for an AI coding assistant such as Claude Code. Use them one at a time, in order. After each prompt, run the tests, read the code, and make sure you understand it before moving on; you will be asked about it in your viva.

How to Use These Prompts

- Give one prompt per conversation turn. Paste the error message back if something fails.
- Ask the assistant to explain anything you do not understand, in simple English.
- After each step run `python -m pytest backend/tests -q` (and `npm run build` for front-end steps).
- Commit to git after every working step so you can go back if something breaks.

Part A: Build the Project from Scratch

Prompt 1 — Project setup

> You are helping me build a final-year project called NeuroMIMO: a 2x2 MIMO-OFDM channel estimation simulator that compares Least Squares (LS), Linear MMSE and a PyTorch deep neural network, with a FastAPI back end and a React dashboard.
>
> Create the folder structure: backend/neuromimo/ with subpackages wireless/, estimation/, evaluation/, storage/ and api/, backend/tests/, backend/models/, backend/results/, and frontend/. Add backend/requirements.txt (fastapi, uvicorn, pydantic, numpy, scipy, torch, scikit-learn, matplotlib, pytest, httpx) and a config.py with dataclasses for SimulationConfig (num_tx=2, num_rx=2, fft_size=64, cp_length=16, pilot_spacing=8, modulation, channel_model, snr_db, num_ofdm_symbols=14, random_seed=42) and enums for modulation (QPSK, 16QAM), channel model (AWGN, RAYLEIGH) and algorithm (PERFECT, LS, MMSE, DNN). Explain each file in simple English.

Prompt 2 — Modulation

> In backend/neuromimo/wireless/modulation.py, implement Gray-coded QPSK and 16-QAM modulators and hard demodulators with unit average symbol energy, a function to generate random bits from a seed, and get_modulator(mod_type). Write pytest tests that check bits -> symbols -> bits is lossless with no noise and that the average symbol energy is 1.

Prompt 3 — OFDM with comb pilots

> In wireless/ofdm.py, implement an OFDMPipeline for 64 subcarriers and a 16-sample cyclic prefix. Use orthogonal comb pilots: transmit antenna 0 uses subcarriers {0,16,32,48}, antenna 1 uses {8,24,40,56}, and each antenna is silent on the other's pilots. Implement IFFT + CP insertion at the transmitter and CP removal + FFT at the receiver, vectorised with NumPy. Add tests that a noiseless, identity channel returns the original grid.

Prompt 4 — Channel models

> In wireless/channel.py, implement a WirelessChannel class with AWGN and a 4-tap Rayleigh multipath channel with an exponential power-delay profile (configurable decay and number of taps), independent for every transmit/receive antenna pair, block fading per frame, reproducible from a seed. Return the true frequency response H with shape (num_rx, num_tx, 64). Add noise at a requested SNR in dB. Test that the average channel power is 1 and that the measured SNR matches the requested SNR.

Prompt 5 — LS and LMMSE estimators

> In estimation/, create an abstract BaseChannelEstimator, then LSEstimator (H_LS = Y/X on pilots, then DFT-based interpolation to all 64 subcarriers) and MMSEEstimator (H = R_HP (R_PP + sigma^2 I)^-1 H_LS using the exact correlation of the exponential power-delay profile). Add tests that LS is exact without noise and that LMMSE beats LS on NMSE at 0 dB SNR.

Prompt 6 — Equalisation and the end-to-end pipeline

> In wireless/equalization.py, implement zero-forcing and MMSE MIMO equalisers. In wireless/simulation.py, build MIMOSimulationPipeline that runs bits -> modulation -> OFDM -> channel -> noise -> OFDM demodulation -> channel estimation -> equalisation -> demodulation, and returns NMSE, BER, SER, execution time and samples for plotting (constellation points and |H|). In evaluation/metrics.py add MSE, NMSE, NMSE in dB, BER, SER, and a 95% Wilson confidence interval with the rule of three when there are zero errors.

Prompt 7 — Residual DNN estimator and training

> In estimation/dnn.py, create ChannelEstimationNet: an MLP 512 -> 256 -> 128 -> 256 -> 512 with LayerNorm, GELU and dropout 0.05, used as a residual: H_DNN = H_LS + f(H_LS). Inputs and outputs are the real and imaginary parts of the 2x2x64 channel flattened to 512 values. In estimation/trainer.py, generate a dataset over SNR 0-30 dB, split it by channel-realisation seed into train/validation/test BEFORE extracting features (no data leakage), and train with Adam, MSE loss, a learning-rate scheduler and best-validation checkpointing to backend/models/neuromimo_dnn.pt. Add a DNNEstimator wrapper and a command-line trainer (train_cli.py).

Prompt 8 — Experiments and figures

> In evaluation/experiments.py, create an ExperimentRunner that runs, on identical channel realisations for every estimator: an SNR sweep 0-30 dB, an AWGN vs Rayleigh comparison, a generalisation test (power-delay-profile decay 0.8/1.5/2.5 and a 6-tap channel), a pilot-spacing study (4/8/16) and an inference-time benchmark. Save JSON and CSV results and 300 DPI Matplotlib figures to backend/results/. Add run_experiments_cli.py with --trials, --quick and --seed options.

Prompt 9 — REST API and database

> In api/main.py, build a FastAPI app with Pydantic schemas and these endpoints: GET /api/health, GET /api/config, POST /api/simulation/run, POST /api/estimation/compare, POST /api/model/train (synchronous or background thread, 409 if a job is already running), GET /api/model/status, GET /api/model/train/status, POST /api/model/predict, GET /api/experiments, GET /api/experiments/{id}, GET /api/experiments/{id}/plots and GET /api/experiments/benchmark/snr-sweep. Store runs in SQLite (storage/database.py). Read allowed CORS origins from NEUROMIMO_CORS_ORIGINS. Write API tests with TestClient, and make sure tests save any trained model to a temporary folder.

Prompt 10 — React dashboard

> Create a Vite + React + TypeScript + Tailwind + Recharts front end in frontend/ with a typed API client (src/api/client.ts) and a Vite proxy from /api to http://127.0.0.1:8000. Build a navbar with API status, a sidebar, and pages: Overview, MIMO-OFDM Simulator, Channel Estimation Lab, ML Model Training (live loss curve), Performance Comparison (NMSE and BER charts, latency, summary cards computed from the data, never hard-coded), Signal Visualization (IQ constellation, channel frequency response, power-delay profile), Experiment History (table with CSV export) and About. Use a dark theme.

Part B: Add New Features

Each prompt assumes the finished project from Part A (or this repository) is open.

Feature 1 — Remove the DNN high-SNR error floor

> In NeuroMIMO, the residual DNN estimator saturates near -25 dB NMSE above 15 dB SNR while LS keeps improving. Add the SNR (in dB, normalised) as an extra input feature to ChannelEstimationNet, retrain, and compare against the current model with the SNR sweep experiment. Keep the old checkpoint loadable. Report the NMSE at 20, 25 and 30 dB before and after.

Feature 2 — 2D CNN estimator

> Add a new estimator, CNNEstimator, that treats the channel as an image over subcarriers and OFDM symbols (in the style of ChannelNet: a super-resolution network followed by a denoising network). Add it to the algorithm enum, the API, the comparison page and all experiments, and add unit tests.

Feature 3 — Doppler and time-varying channels

> Extend WirelessChannel with a time-varying Rayleigh channel using the Jakes Doppler spectrum, configurable by maximum Doppler frequency (for example 0, 100, 300 Hz) and subcarrier spacing. Add a Doppler experiment that plots NMSE and BER versus Doppler for all estimators at 20 dB SNR, and a Doppler slider on the Simulator page.

Feature 4 — Larger MIMO configurations

> Generalise the simulator, estimators and DNN input size from 2x2 to NxM MIMO (support 2x2, 4x4 and 8x8) with orthogonal comb pilots for every transmit antenna. Update the API schema, the antenna selector on the Simulator page, and add tests for 4x4.

Feature 5 — 16-QAM and 64-QAM DNN models

> Train and store separate DNN checkpoints per modulation (QPSK, 16-QAM, 64-QAM; add 64-QAM to the modulation module). Let the API pick the checkpoint that matches the request, and show which model was used on the Channel Estimation Lab page.

Feature 6 — Learned pilot placement

> Add an experiment that compares fixed comb pilots with a pilot pattern chosen by a simple search (for example, greedy selection that minimises validation NMSE) for 2, 4 and 8 pilots per antenna. Plot NMSE versus number of pilots for LS, LMMSE and the DNN.

Feature 7 — PDF report export

> Add a 'Download report' button on the Performance Comparison page that calls a new endpoint GET /api/report.pdf. The endpoint builds a PDF (with ReportLab) containing the summary cards, the NMSE and BER tables and the figures from backend/results/.

Feature 8 — Docker and one-command start

> Add a Dockerfile for the back end, a Dockerfile for the front end (build with Vite, serve with nginx, proxy /api to the back end) and a docker-compose.yml so `docker compose up` starts the whole app on http://localhost:8080. Document it in the README.

Feature 9 — Continuous integration

> Add a GitHub Actions workflow that, on every push and pull request, installs Python 3.12 dependencies and runs pytest on backend/tests, then installs Node 22 and runs npm ci, npm run lint and npm run build in frontend/. Cache pip and npm.

Feature 10 — Hyperparameter search page

> Add a page that launches a small grid search over learning rate, batch size and hidden layer sizes for the DNN as a background job, streams progress like the training page, and shows a sortable results table with the best configuration highlighted.